



Qualcomm Technologies, Inc.

Qualcomm Linux Graphics Guide

80-70014-19 Rev. AK

July 12, 2024

Contents

- 1 Overview 4
- 2 Getting started 7
 - 2.1 OpenGL ES sample application 8
 - 2.2 OpenCL sample application 11
 - 2.3 Vulkan sample applications 13
 - 2.3.1 Sascha Willems Vulkan sample application 13
 - 2.3.2 Khronos Vulkan sample application 14
- 3 Features 17
- 4 Tools 18
- 5 Debug 19
 - 5.1 Weston/Wayland logs 19
 - 5.2 Kernel/KGSL 19
 - 5.3 OpenGL ES applications 21
 - 5.4 OpenCL applications 21
 - 5.5 Vulkan applications 21
- 6 References 23
 - 6.1 Related documents 23
 - 6.2 Acronyms and terms 23

Tables

Table 3-1: Adreno GPU specifications..... 17

1 Overview

The graphics subsystem is powered by the Qualcomm® Adreno™ GPU. The Adreno GPU is a three-dimensional (3D) graphics accelerator with 64-bit addressing and incorporates graphics memory (GMEM). The GMEM serves as a dedicated memory for the graphics subsystem and facilitates Fast Z, color, and stencil rendering.

Additionally, the Adreno GPU supports the Qualcomm® Universal Bandwidth Compression, enhancing the overall performance.

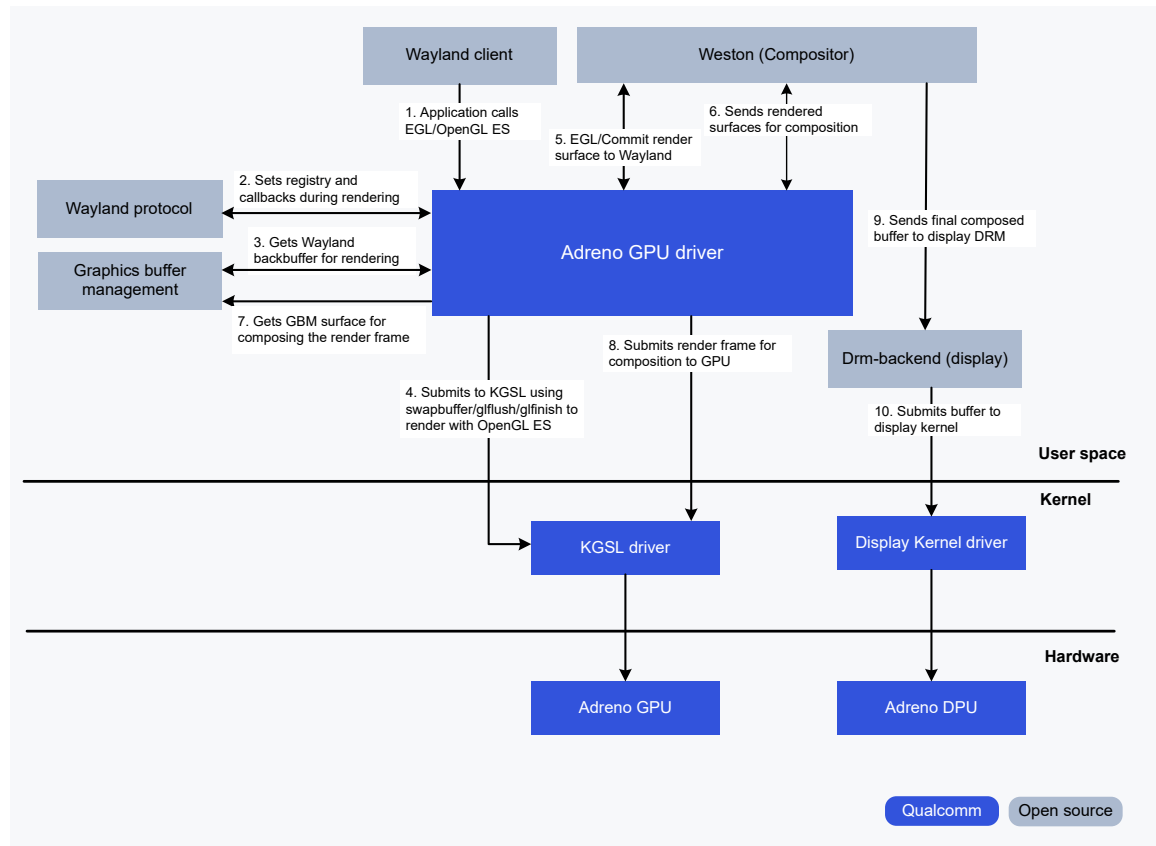
The Adreno GPU supports the following graphics and compute APIs:

- **OpenGL ES:** [3.0](#), [3.1](#), [3.2](#)
- **OpenCL:** [1.2 FP](#), [2.0 FP](#)
- **EGL:** [1.5](#)
- **Vulkan:** [1.1](#)

NOTE The Qualcomm Linux platform allows you to develop applications for QCS6490 and QCS5430.

Architecture

The following figure shows the architecture of the graphics subsystem.



Adreno GPU driver

The Adreno GPU driver comprises a collection of user-mode precompiled libraries and firmware developed by Qualcomm. The Adreno GPU driver must be used as-is.

NOTE To use all the features supported by the Adreno GPU, ensure that the Adreno GPU driver libraries and firmware listed in the following tables are present on the device at `/lib/firmware/`.

Adreno GPU libraries

The following table lists the closed-source Adreno graphics libraries supported in the user-mode driver:

Library	Description
libCB.so	Driver for GPU computing
libEGL_adreno.so	EGL 1.4 driver
libeglSubDriverWayland.so	EGL driver for Wayland subsystem
libadreno_utils.so	Utility library for Adreno GPU driver
libgsl.so	GSL library
libq3dtools_adreno.so	Adreno profiler support layer

Library	Description
libGLESv1_CM_adreno.so	OpenGL ES 1.1 driver
libGLESv2_adreno.so	OpenGL ES 2.0/3.0/3.1 driver
libq3dtools_esx.so	OpenGL ES profiler support layer
libOpenCL.so	OpenCL library
libOpenCL_adreno.so	OpenCL library for Adreno GPU
libllvm-qcom.so	OpenCL compiler library
libvulkan_adreno.so	Vulkan 1.x driver
libllvm-glnext.so	OpenGL ES and Vulkan program binary loader
libllvm-qgl.so	OpenGL ES and Vulkan core compiler library

Adreno GPU firmware

The following table lists the firmware files of the Adreno GPU hardware:

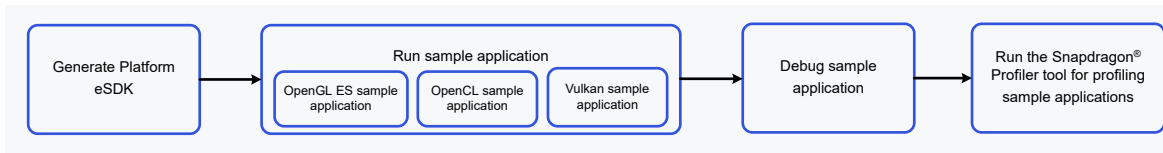
Firmware	Description
a660_sqe.fw	Microcode for command processor
a660_zap	Secure microcode for secure content support
a660_gmu.bin	Microcode for graphics management unit (GMU)

Kernel Graphics Support Layer (KGSL)

The KGSL serves as the kernel-mode driver for the Adreno GPU. Its primary function is to submit commands generated by the Adreno user-mode driver to the Adreno GPU for processing. Additionally, the KGSL communicates with the graphics management unit (GMU) to ensure proper state management. The following table provides details about the KGSL driver:

Chipset	QCS6490/QCS5430
Linux kernel version	6.6
Source code path	<root_dir>/source/graphics-kernel/
GPU DTSI filepath	■ QCS6490: □ <root_dir>/source/graphics-devicetree/gpu/qcm6490-graphics.dtsi □ <root_dir>/source/graphics-devicetree/gpu/qcm6490-graphics-pwrlevels.dtsi ■ QCS5430: □ <root_dir>/source/graphics-devicetree/gpu/qcs5430-graphics.dtsi □ <root_dir>/source/graphics-devicetree/gpu/qcs5430-graphics-pwrlevels.dtsi

2 Getting started



Before compiling any sample application, generate Platform eSDK using one of the following methods:

- Generate Platform eSDK using Qualcomm release archive
- Generate Platform eSDK by manually compiling the LE.QCLINUX.1.0 SDK

Generate Platform eSDK using Qualcomm release archive

1. Download the eSDK from the Qualcomm release archive.

- For Ubuntu x86 architecture-based host machines:

```
wget https://artifacts.codeinaro.org/artifactory/qli-ci/flashable-binaries/qimpsdk/qcm6490/x86/qcom-6.6.28-QLI.1.1-Ver.1.1_qim-product-sdk-1.1.3.zip
```

- For Arm architecture-based host machines:

```
wget https://artifacts.codeinaro.org/artifactory/qli-ci/flashable-binaries/qimpsdk/qcm6490/arm/qcom-6.6.28-QLI.1.1-Ver.1.1_qim-product-sdk-1.1.3.zip
```

2. Unzip the SDK to a folder of your choice:

```
unzip qcom-6.6.28-QLI.1.1-Ver.1.1_qim-product-sdk-1.1.3.zip
```

After unzipping, verify that the platform eSDK installer `qcom-wayland-x86_64-qcom-multimedia-image-armv8-2a-qcm6490-toolchain-ext-1.0.sh` is present at:
`<unzip_location>/target/qcm6490/sdk/`.

3. Run the installer script.

```
umask a+rx
```

```
sh ./qcom-wayland-x86_64-qcom-multimedia-image-armv8-2a-qcm6490-toolchain-ext-1.0.sh
```

4. Follow the instructions on the console to install the Platform eSDK on your host PC.

For more information on Platform eSDK, see [Download QIM Platform eSDK](#).

Generate Platform eSDK by manually compiling LE.QCLINUX.1.0 SDK

1. Compile LE.QCLINUX.1.0 SDK and verify that the installer (qcom-wayland-x86_64-qcom-multimedia-image-armv8-2a-qcm6490-toolchain-ext-1.0.sh) is generated at <root directory>/build-qcom-wayland/tmp-glibc/deploy/sdk.

2. Run the installer script.

```
umask a+rx
```

```
sh ./qcom-wayland-x86_64-qcom-multimedia-image-armv8-2a-qcm6490-toolchain-ext-1.0.sh
```

3. Follow the instructions on the console to install the Platform eSDK on your host PC.

For build compilation and SDK creation procedures, see [Qualcomm Linux Build Guide](#).

2.1 OpenGL ES sample application

Supported EGL client extensions

EGL_EXT_client_extensions, EGL_KHR_client_get_all_proc_addresses EGL_EXT_platform_base, EGL_KHR_platform_android EGL_KHR_platform_wayland, EGL_KHR_platform_gbm, EGL_KHR_platform_x11, OpenGL_ES, EGL_KHR_image EGL_KHR_image_base, EGL_QCOM_create_image EGL_KHR_lock_surface, EGL_KHR_lock_surface2 EGL_KHR_lock_surface3, EGL_KHR_gl_texture_2D_image, EGL_KHR_gl_texture_cubemap_image, EGL_KHR_gl_texture_3D_image, EGL_KHR_gl_renderbuffer_image, EGL_ANDROID_blob_cache EGL_KHR_create_context, EGL_KHR_surfaceless_context, EGL_KHR_create_context_no_error, EGL_KHR_get_all_proc_addresses, EGL_QCOM_lock_image2, EGL_KHR_no_config_context EGL_EXT_surface_SMPTE2086_metadata, EGL_EXT_image_dma_buf_import, EGL_EXT_image_dma_buf_import_modifiers EGL_EXT_yuv_surface, EGL_IMG_context_priority, EGL_WL_bind_wayland_display, EGL_WL_create_wayland_buffer_from_image, EGL_ANDROID_native_fence_sync, EGL_EXT_create_context_robustness, EGL_KHR_fence_sync, EGL_KHR_wait_sync, EGL_KHR_mutable_render_buffer EGL_KHR_partial_update, EGL_KHR_surfaceless_context

Supported GL extensions

```
GL_OES_EGL_image GL_OES_EGL_image_external, GL_OES_EGL_sync GL_OES_vertex_half_float,
GL_OES_framebuffer_object GL_OES_rgb8_rgba8, GL_OES_compressed_ETC1_RGB8_texture,
GL_AMD_compressed_ATC_texture, GL_KHR_texture_compression_astc_ldr,
GL_KHR_texture_compression_astc_hdr, GL_OES_texture_compression_astc
GL_EXT_texture_compression_s3tc, GL_EXT_texture_compression_s3tc_srgb,
GL_EXT_texture_compression_rgtc GL_EXT_texture_compression_bptc, GL_OES_texture_npot,
GL_EXT_texture_filter_anisotropic, GL_EXT_texture_format_BGRA8888 GL_EXT_read_format_bgra,
GL_OES_texture_3D GL_EXT_color_buffer_float, GL_EXT_color_buffer_half_float GL_EXT_float_blend,
GL_QCOM_alpha_test, GL_OES_depth24 GL_OES_packed_depth_stencil, GL_OES_depth_texture
GL_OES_depth_texture_cube_map GL_EXT_sRGB, GL_OES_texture_float GL_OES_texture_float_linear,
GL_OES_texture_half_float, GL_OES_texture_half_float_linear, GL_EXT_texture_type_2_10_10_10_REV
GL_EXT_texture_sRGB_decode, GL_EXT_texture_compression_astc_decode_mode,
GL_EXT_texture_mirror_clamp_to_edge, GL_EXT_texture_format_sRGB_override
GL_OES_element_index_uint GL_EXT_copy_image GL_EXT_geometry_shader,
GL_EXT_tessellation_shader GL_OES_texture_stencil8, GL_EXT_shader_io_blocks
GL_OES_shader_image_atomic, GL_OES_sample_variables GL_EXT_texture_border_clamp,
GL_EXT_EGL_image_external_wrap_modes, GL_EXT_multisampled_render_to_texture,
GL_EXT_multisampled_render_to_texture2, GL_OES_shader_multisample_interpolation,
GL_EXT_texture_cube_map_array GL_EXT_draw_buffers_indexed, GL_EXT_gpu_shader5
GL_EXT_robustness, GL_EXT_texture_buffer, GL_EXT_shader_framebuffer_fetch,
GL_ARM_shader_framebuffer_fetch_depth_stencil, GL_OES_texture_storage_multisample_2d_array,
GL_OES_sample_shading GL_OES_get_program_binary, GL_EXT_debug_label
GL_KHR_blend_equation_advanced, GL_KHR_blend_equation_advanced_coherent
GL_QCOM_tiled_rendering, GL_ANDROID_extension_pack_es31a GL_EXT_primitive_bounding_box,
GL_OES_standard_derivatives GL_OES_vertex_array_object, GL_EXT_disjoint_timer_query
GL_KHR_debug, GL_EXT_YUV_target, GL_EXT_sRGB_write_control GL_EXT_texture_norm16,
GL_EXT_discard_framebuffer, GL_OES_surfaceless_context, GL_OVR_multiview GL_OVR_multiview2
GL_EXT_texture_sRGB_R8, GL_KHR_no_error GL_EXT_debug_marker,
GL_OES_EGL_image_external_essl3, GL_OVR_multiview_multisampled_render_to_texture,
GL_EXT_buffer_storage GL_EXT_external_buffer, GL_EXT_blit_framebuffer_params
GL_EXT_clip_cull_distance, GL_EXT_shader_non_constant_global_initializers, GL_QCOM_texture_foveated
GL_QCOM_texture_foveated2, GL_QCOM_texture_foveated_subsampled_layout,
GL_QCOM_shader_framebuffer_fetch_noncoherent, GL_QCOM_shader_framebuffer_fetch_rate
GL_EXT_memory_object, GL_EXT_memory_object_fd GL_EXT_EGL_image_array,
GL_NV_shader_noperspective_interpolation, GL_KHR_robust_buffer_access_behavior
GL_EXT_EGL_image_storage, GL_EXT_blend_func_extended GL_EXT_clip_control, GL_OES_texture_view
GL_EXT_fragment_invocation_density, GL_QCOM_motion_estimation, GL_QCOM_YUV_texture_gather,
GL_QCOM_shading_rate GL_IMG_texture_filter_cubic, GL_EXT_polygon_offset_clamp
GL_EXT_texture_sRGB_RG8, GL_EXT_shader_implicit_conversions GL_EXT_render_snorm,
GL_QCOM_render_sRGB_R8_RG8
```

Prerequisites:

- Generate the Platform eSDK. For instructions, see [Getting started](#).
- Enable SSH in Permissive mode to securely access your host device. For instructions, see [How to SSH](#).

glmark2 is an OpenGL ES based sample application provided in the Platform eSDK. Following are the steps to compile and run the glmark2 application:

1. Compile glmark2 application:

- a. Set the SDK environment in the Linux terminal.

```
source environment-setup-armv8-2a-qcom-linux
```

- b. Compile and generate a binary.

```
devtool modify glmark2
```

```
devtool build glmark2
```

If there is any umask error, set umask 022.

Upon successful compilation, the output files are generated at <path-to-installed-eSDK>/workspace/sources/glmark2/oe-workdir/image/usr/bin.

2. Push the glmark2 binary to the device.

```
scp -r <path-to-installed-Platform-eSDK>/workspace/sources/glmark2/oe-workdir/image/usr/bin/glmark2-es2-wayland root@[IP-address-of-device]:/tmp
scp -r <path-to-installed-Platform-eSDK>/workspace/sources/glmark2/oe-workdir/image/usr/share/glmark2 root@[IP-address-of-device]:/tmp
```

3. Run the glmark2 application on the device using SSH.

a. Open the SSH terminal using the device IP address.

b. Run the following commands:

```
setenforce 0
mount -o remount,rw /
export XDG_RUNTIME_DIR=/dev/socket/weston export
WAYLAND_DISPLAY=wayland-1
chmod 777 /tmp/glmark2-es2-wayland
chmod -R 777 /tmp/glmark2
cd /tmp
./glmark2-es2-wayland --data-path /tmp/glmark2 -b jellyfish
```

NOTE Before executing the glmark2 application, ensure that the Weston application is running. To launch Weston, run the following commands:

```
setenforce 0
. /etc/profile
export XDG_RUNTIME_DIR=/dev/socket/weston
mkdir --parents $XDG_RUNTIME_DIR
chmod 0700 $XDG_RUNTIME_DIR
cd /usr/bin/
killall weston
weston --tty=2 --idle-time=0 --log=/tmp/weston.log --
config=/etc/xdg/weston/weston.ini --continue-without-input
```

For more information about the Weston application, see [Qualcomm Linux Display Guide](#).

2.2 OpenCL sample application

Supported OpenCL extensions

```
cl_khr_icd, cl_img_egl_image, cl_khr_3d_image_writes, cl_khr_byte_addressable_store,
cl_khr_depth_images, cl_khr_egl_event, cl_khr_egl_image, cl_khr_fp16, cl_khr_gl_sharing,
cl_khr_global_int32_base_atomics, cl_khr_global_int32_extended_atomics, cl_khr_image2d_from_buffer,
cl_khr_local_int32_base_atomics, cl_khr_local_int32_extended_atomics, cl_khr_mipmap_image,
cl_khr_srgb_image_writes, cl_khr_subgroups, cl_qcom_accelerated_image_ops,
cl_qcom_compressed_image, cl_qcom_compressed_yuv_image_read, cl_qcom_create_buffer_from_image,
cl_qcom_dot_product8, cl_qcom_ext_host_ptr,
cl_qcom_ext_host_ptr_io coherent, cl_qcom_extended_query_image_info, cl_qcom_extract_image_plane,
cl_qcom_dmabuf_host_ptr, cl_qcom_other_image, cl_qcom_perf_hint, cl_qcom_priority_hint,
cl_qcom_protected_context, cl_qcom_recordable_queues, cl_qcom_reqd_sub_group_size,
cl_qcom_subgroup_shuffle, cl_qcom_vector_image_op
```

Prerequisites:

- Generate the Platform eSDK. For instructions, see [Getting started](#).
- Enable SSH in Permissive mode to securely access your host device. For instructions, see [How to SSH](#).
- Download the Adreno OpenCL [SDK 2.0](#).

NOTE You must be a registered user to download the Adreno OpenCL SDK.

Multiple sample applications are provided in the Adreno OpenCL SDK. The steps to compile and run any application remain the same. Following are the steps to compile and run the hello world sample application:

1. Compile the application on the host machine.

```
cd openc1-sdk

source <path-to-installed-Platform-eSDK>/environment-setup-armv8-2a-qcom-
linux

cmake -B build -DCLSDK_OPENCL_LIBRARY=$OECORE_TARGET_SYSROOT/usr/lib/
libOpenCL.so -DCLSDK_DMABUFHEAP_LIBRARY=$OECORE_TARGET_SYSROOT/usr/lib/
libdmabufheap.so -DOPEN_EMBEDDED=1

cmake --build build
```

After successful compilation, binary files are generated in the `build` directory.

2. Run the application:

- a. Open the SSH terminal using the device IP address.
- b. Run the following commands on the device using the SSH terminal:

```
setenforce 0

mount -o remount,rw /

mkdir -p /opt/data

mkdir -p /opt/data/openc1
```

- c. Run the following commands on the host machine:

```
scp -r <opencl-sdk>/build/* root@[IP-address-of-the device]:/opt/data/  
opencl/
```

```
scp -r <opencl-sdk>/example_images/* root@[IP-address-of-the  
device]:/opt/data/opencl/
```

- d. Run the following commands on the device using the SSH terminal:

```
cd /opt/data/opencl/
```

```
mkdir out
```

```
chmod 777 ./*
```

```
echo "run hello world 2.0 opencl sdk" > hello_world_input.txt
```

```
touch out/hello_world_output.txt
```

```
cat out/hello_world_output.txt
```

```
./cl_sdk_hello_world hello_world_input.txt out/hello_world_output.txt
```

```
cat out/hello_world_output.txt
```

2.3 Vulkan sample applications

Supported Vulkan extensions

```

VK_KHR_get_physical_device_properties2, VK_KHR_surface, VK_KHR_external_semaphore_capabilities,
VK_KHR_external_memory_capabilities, VK_KHR_device_group_creation, VK_EXT_debug_utils,
VK_KHR_wayland_surface, VK_KHR_external_fence_capabilities, VK_KHR_get_surface_capabilities2,
VK_EXT_debug_report, VK_EXT_subgroup_size_control, VK_KHR_external_memory,
VK_EXT_pipeline_creation_feedback, VK_KHR_shader_float16_int8, VK_KHR_get_memory_requirements2,
VK_KHR_copy_commands2, VK_KHR_spirv_1_4, VK_EXT_fragment_density_map,
VK_KHR_external_semaphore_fd, VK_KHR_swapchain, VK_QCOM_render_pass_store_ops,
VK_EXT_astc_decode_mode, VK_KHR_shared_presentable_image, VK_KHR_external_memory_fd,
VK_QCOM_render_pass_shader_resolve, VK_KHR_maintenance1, VK_KHR_maintenance2,
VK_KHR_maintenance3, VK_KHR_separate_depth_stencil_layouts, VK_EXT_image_robustness,
VK_KHR_buffer_device_address, VK_EXT_extended_dynamic_state, VK_EXT_queue_family_foreign,
VK_KHR_bind_memory2, VK_KHR_external_semaphore, VK_KHR_shader_terminate_invocation,
VK_QCOM_fragment_density_map_offset, VK_EXT_scalar_block_layout,
VK_KHR_sampler_ycbcr_conversion, VK_EXT_vertex_attribute_divisor, VK_KHR_variable_pointers,
VK_QCOM_multiview_per_view_viewports, VK_KHR_push_descriptor, VK_KHR_timeline_semaphore,
VK_EXT_device_memory_report, VK_KHR_imageless_framebuffer, VK_KHR_device_group,
VK_EXT_device_fault, VK_KHR_relaxed_block_layout, VK_KHR_external_fence,
VK_KHR_shader_non_semantic_info, VK_EXT_shader_atomic_float, VK_EXT_custom_border_color,
VK_EXT_host_query_reset, VK_EXT_index_type_uint8, VK_KHR_multiview,
VK_KHR_storage_buffer_storage_class, VK_EXT_image_drm_format_modifier,
VK_EXT_fragment_density_map2, VK_QCOM_rotated_copy_commands,
VK_KHR_shader_subgroup_extended_types, VK_EXT_private_data,
VK_EXT_pipeline_creation_cache_control, VK_EXT_robustness2, VK_EXT_shader_module_identifier,
VK_EXT_global_priority_query, VK_EXT_separate_stencil_usage, VK_EXT_vertex_input_dynamic_state,
VK_IMG_filter_cubic, VK_EXT_filter_cubic, VK_QCOM_tile_properties, VK_KHR_image_format_list,
VK_EXT_external_memory_dma_buf, VK_EXT_sampler_filter_minmax, VK_KHR_16bit_storage,
VK_KHR_pipeline_executable_properties, VK_EXT_shader_demote_to_helper_invocation,
VK_QCOM_render_pass_transform, VK_KHR_create_renderpass2, VK_EXT_transform_feedback,
VK_EXT_blend_operation_advanced, VK_EXT_provoking_vertex,
VK_QCOM_multiview_per_view_render_areas, VK_KHR_depth_stencil_resolve,
VK_KHR_shader_float_controls, VK_EXT_texture_compression_astc_hdr, VK_EXT_global_priority,
VK_KHR_shader_draw_parameters, VK_KHR_vulkan_memory_model, VK_EXT_descriptor_indexing,
VK_EXT_depth_clip_enable, VK_KHR_synchronization2, VK_EXT_line_rasterization,
VK_KHR_fragment_shading_rate, VK_KHR_descriptor_update_template, VK_KHR_draw_indirect_count,
VK_KHR_driver_properties, VK_KHR_uniform_buffer_standard_layout, VK_KHR_dedicated_allocation,
VK_EXT_primitive_topology_list_restart, VK_KHR_global_priority, VK_EXT_sample_locations,
VK_KHR_sampler_mirror_clamp_to_edge, VK_KHR_external_fence_fd

```

Prerequisites:

- Generate the Platform eSDK. For instructions, see [Getting started](#).
- Enable SSH in Permissive mode to securely access your host device. For instructions, see [How to SSH](#).

Multiple sample applications are provided in the Adreno SDK for Vulkan. The steps to compile and run any application remain the same. As an example, the steps to run a Sascha Willems and Khronos Vulkan sample applications are described here.

2.3.1 Sascha Willems Vulkan sample application

The Sascha Willems Vulkan sample applications are available at: <https://github.com/SaschaWillems/Vulkan.git>. The steps to compile and run any Sascha Willems Vulkan application are the same.

Following are the steps to compile and run the triangle bin application:

1. Compile the application.

```
git clone --recurse-submodules https://github.com/SaschaWillems/Vulkan.git
cd Vulkan

source <path-to-installed-Platform-eSDK>/environment-setup-armv8-2a-qcom-
linux

cmake -G "Unix Makefiles" -Bbuild/linux -DUSE_WAYLAND_WSI=ON -
DRESOURCE_INSTALL_DIR="/tmp/"

cmake --build build/linux --config Release -j$(nproc)
```

After successful compilation, binary files are generated at `build/linux/bin`.

2. Run the application:

a. Run the following commands on the host machine:

```
cd Vulkan

scp -r assets root@[IP-address-of-device]:/tmp/
scp -r shaders root@[IP-address-of-device]:/tmp/
scp -r build/linux/bin/triangle root@[IP-address-of-device]:/tmp/
```

b. Open the SSH terminal using the device IP address and run the following commands:

```
cd /tmp

chmod -R 777 assets
chmod -R 777 shaders
chmod 777 triangle

. /etc/profile

export XDG_RUNTIME_DIR=/dev/socket/weston
mkdir --parents $XDG_RUNTIME_DIR
chmod 0700 $XDG_RUNTIME_DIR
export WAYLAND_DISPLAY=wayland-1

./triangle
```

2.3.2 Khronos Vulkan sample application

The Khronos Vulkan sample applications are available at: <https://github.com/KhronosGroup/Vulkan-Samples.git>. The steps to compile and run any Khronos Vulkan application are the same.

Following are the steps to compile and run the `vulkan_samples` application:

1. To compile the application, run the following commands on the host machine:

```
sudo apt-get install -y libwayland-dev

git clone --recurse-submodules https://github.com/KhronosGroup/Vulkan-
Samples.git

cd Vulkan-Samples/

source <path-to-installed-Platform-eSDK>/environment-setup-armv8-2a-qcom-
linux

cmake -G "Unix Makefiles" -Bbuild/linux -DCMAKE_BUILD_TYPE=Release -
DVKB_WSI_SELECTION=WAYLAND -DGLFW_BUILD_X11=OFF
```

If the compilation fails, do the following:

- a. Open `Vulkan-Samples/build/linux/CmakeCache.txt` file.
 - b. Change `GLFW_BUILD_X11` to `OFF`.
2. Generate the binary files for the target device:

```
cmake --build build/linux --config Release --target vulkan_samples -j$
(nproc)
```

After successful compilation, binary files are generated at `Vulkan-Samples/build/linux/app/bin/Release/aarch64`.

3. Run the application:

- a. Run the following commands on the host machine:

```
cd Vulkan-Samples

scp -r assets root@[IP-address-of-device]:/tmp/

scp -r shaders root@[IP-address-of-device]:/tmp/

scp -r build/linux/app/bin/Release/aarch64/vulkan_samples root@[IP-
address-of-device]:/tmp/
```

- b. Open the SSH terminal using the device IP address and run the following commands:

```
cd /tmp
chmod -R 777 assets
chmod -R 777 shaders
chmod 777 vulkan_samples
. /etc/profile
export XDG_RUNTIME_DIR=/dev/socket/weston
mkdir --parents $XDG_RUNTIME_DIR
chmod 0700 $XDG_RUNTIME_DIR
export WAYLAND_DISPLAY=wayland-1
./vulkan_samples sample swapchain_images
```

NOTE If you are unable to run these sample applications from the `/tmp` location due to space issues, try alternative locations such as `/etc`.

3 Features

The following table lists the specifications of the Adreno GPU:

Table 3-1 Adreno GPU specifications

Chipset	QCS6490	QCS5430
GPU	Adreno GPU 643	Adreno GPU 642 L
GPU clock	812 MHz	315 MHz
On-Chip memory (GMEM)	512 KB	512 KB

For general information about the Adreno GPU, see the following:

- [Game Developer Guides](#)
- [Best practices](#)
- [Frequently asked questions](#)

4 Tools

Qualcomm provides the Snapdragon[®] Profiler as a performance profiling tool for the Adreno GPU.

Snapdragon Profiler

The Snapdragon Profiler is a system profiling tool. It enables you to examine various aspects such as CPU, GPU, DSP, memory, power, thermal, and network metrics to identify and resolve performance issues. The Snapdragon Profiler offers the following features:

- Real-time and Trace modes for the OpenGL ES, Vulkan, and OpenCL applications
- Snapshot mode for the OpenGL ES and Vulkan applications

Download the latest version of the [Snapdragon Profiler](#) and for more information, see [Snapdragon Profiler Quick Start Guide](#).

NOTE Ensure that you are logged in to the Qualcomm developer network (QDN).

If you do not have ADB configured in your path, in Snapdragon Profiler, go to **File > Settings** and set the ADB path on the **Android** tab.

5 Debug

For performance profiling of the applications, you can use Snapdragon Profiler and kernel commands. The procedures to collect information for profiling and debugging using the kernel commands, and also how to collect logs are described here.

NOTE Use the commands described here carefully, as these commands may impact the power consumption of the device.

5.1 Weston/Wayland logs

To capture the Weston/Wayland debug logs, run the following command before launching the Weston or any application:

```
export WAYLAND_DEBUG=server/client
```

5.2 Kernel/KGSL

Before debugging the kernel or KGSL, open the SSH terminal using the device IP address and run the following commands:

```
setenforce 0  
mount -o remount rw /
```

Set CPU to performance mode

To set the CPU to performance mode, do the following:

1. Set all CPU cores by replacing `cpux` with `cpu0`, `cpu1`, and so on.

```
echo 1 > /sys/devices/system/cpu/[cpux]/online  
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_governors
```

Sample output:

```
ondemand userspace performance schedutil
```

2. Run the command:

```
echo performance > /sys/devices/system/cpu/[cpux]/cpufreq/scaling_governor
```

3. Validate the trial settings:

```
cat /sys/devices/system/cpu/[cpux]/online (cpux : cpu0,cpu1 ...)  
cat /sys/devices/system/cpu/[cpux]/cpufreq/scaling_governor (cpux : cpu0,  
cpu1 ...)
```

Set GPU frequency

1. For the power level, check the `available_frequencies` sys node.

```
cat /sys/class/kgsl/kgsl-3d0/devfreq/available_frequencies
```

Sample output:

```
812000000 700000000 608000000 550000000 450000000 315000000
```

From the sample output, GPU power levels are interpreted as following:

Frequency	812000000	700000000	608000000	550000000	450000000	315000000
Power level	0	1	2	3	4	5

2. Increment the GPU power level by one and perform the GPU power-level test by substituting `<level>` with 0, 1, 2, 3, 4, and so on, in the following commands:

```
echo <level> > /sys/class/kgsl/kgsl-3d0/min_pwrlevel
```

```
echo <level> > /sys/class/kgsl/kgsl-3d0/max_pwrlevel
```

NOTE The Adreno GPU frequency for QCS5430 is fixed at 315 MHz. For QCS5430, the power levels mentioned here are not supported and the output of the `available_frequencies` command will be 315000000.

Set GPU to performance mode

NOTE For QCS5430, the performance mode is not supported as the frequency of the Adreno GPU is fixed at 315 MHz.

- To set the GPU to performance mode, run the following commands. These commands force the GPU to always run at the maximum frequency.

```
echo 0 > /sys/class/kgsl/kgsl-3d0/min_pwrlevel
```

```
echo 0 > /sys/class/kgsl/kgsl-3d0/max_pwrlevel
```

- Run the following command to ensure that the GPU does not enter the low-power state even if the device is idle for 10000000 ms.

```
echo 10000000 > /sys/class/kgsl/kgsl-3d0/idle_timer
```

- To set the GPU to performance mode while ensuring that the GPU does not enter the low-power state, run the following commands:

```
echo 0 > /sys/class/kgsl/kgsl-3d0/min_pwrlevel
```

```
echo 0 > /sys/class/kgsl/kgsl-3d0/max_pwrlevel
```

```
echo 10000000 > /sys/class/kgsl/kgsl-3d0/idle_timer
```

Check other KGSL settings

- To enable the KGSL clock, run the following command:

```
echo 1 > /sys/class/kgsl/kgsl-3d0/force_clk_on
```

- To enable the KGSL rail, run the following command:

```
echo 1 > /sys/class/kgsl/kgsl-3d0/force_rail_on
```

- To enable the KGSL bus, run the following command:

```
echo 1 > /sys/class/kgsl/kgsl-3d0/force_bus_on
```

5.3 OpenGL ES applications

You can use the Snapdragon Profiler to capture snapshot and real-time traces of OpenGL ES applications. For more information on the Snapdragon Profiler, see [Snapdragon Profiler](#).

5.4 OpenCL applications

You can use the Snapdragon Profiler to capture snapshot and real-time traces of OpenCL applications. For more information on the Snapdragon Profiler, see [Snapdragon Profiler](#).

For performance profiling of OpenCL applications based on events, along with the Snapdragon Profiler you can also see the Performance profiling section in the [Qualcomm® Snapdragon™ Mobile Platform OpenCL General Programming and Optimization Guide](#).

NOTE Some sections in the [Qualcomm® Snapdragon™ Mobile Platform OpenCL General Programming and Optimization Guide](#) are applicable only for Android platforms and not applicable to the Qualcomm Linux platforms. However, some of the profiling options are purely based on the OpenCL API and are therefore supported on the Qualcomm Linux devices. For example, the `clGetEventProfilingInfo` API.

5.5 Vulkan applications

Prerequisites:

You can use the Snapdragon Profiler to capture snapshot and real-time traces of Vulkan applications. For more information on the Snapdragon Profiler, see [Snapdragon Profiler](#). Along with the Snapdragon Profiler, you can also use validation layers to debug Vulkan applications.

To debug a Vulkan application using the validation layers, do the following:

1. Set the source environment.

```
source <path-to-installed-platform-eSDK>/environment-setup-armv8-2a-qcom-  
linux
```

2. Download the validation layers and run the following commands:

```
git clone https://github.com/KhronosGroup/Vulkan-ValidationLayers

cmake -S . -B build -D UPDATE_DEPS=ON -D BUILD_WSI_XLIB_SUPPORT=OFF -D
BUILD_TESTS=OFF -D CMAKE_BUILD_TYPE=Release -D BUILD_WSI_XCB_SUPPORT=OFF -
D BUILD_WSI_WAYLAND_SUPPORT=ON

cmake --build build --config Debug

scp -r build/layers/libVkLayer_khronos_validation.so root@[IP-address-of-
device]:/tmp/local/debug/vulkan/

scp -r build/layers/VkLayer_khronos_validation.json root@[IP-address-of-
device]:/tmp/local/debug/vulkan/

export VK_LAYER_PATH=/tmp/local/debug/vulkan:$VK_LAYER_PATH
```

NOTE If /tmp/local/debug/vulkan path is not available, create similar path.

3. To force the layer to be enabled for Vulkan applications, set the VK_INSTANCE_LAYERS environment variable.

```
export VK_INSTANCE_LAYERS=VK_LAYER_KHRONOS_validation
```

4. Edit or append the following settings in the layers/vk_layer_settings.txt file and push to the device.

```
khronos_validation.debug_action = VK_DBG_LAYER_ACTION_LOG_MSG

khronos_validation.report_flags = error,warn,perf

khronos_validation.file = true

khronos_validation.log_filename = vk_validatedump.txt

scp -r vk_layer_settings.txt root@[IP-address-of-device]:/tmp/local/debug/
vulkan/
```

5. To dump the log, add the path of the validation layers.

```
export VK_LAYER_SETTINGS_PATH=/tmp/local/debug/vulkan/vk_layer_settings.txt
```

Dumps are generated in the same directory where the Vulkan application binary was pushed.

NOTE To avoid data flush in the tmp folder, you can consider pushing the Vulkan application binary to any other suitable path. If you are changing the path, ensure that after the reboot, the path is correctly set in the environment variables.

6 References

6.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
Qualcomm Linux Build Guide	80-70014-254
Qualcomm Linux Display Guide	80-70014-18
Resources	
Snapdragon Profiler Quick Start Guide	
https://www.khronos.org/opengl/	
https://www.khronos.org/opencl/	
https://docs.vulkan.org/guide/latest/vulkan_spec.html	

6.2 Acronyms and terms

Acronym	Definition
GMEM	Graphics memory
GMU	Graphics management unit
KGSL	Kernel graphics support layer
QDN	Qualcomm developer network

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com or the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.